1 Simply Typed Lambda Calculus

1.1 Syntax

Taking inspiration from [1] and [2] we define the types of the simply typed lambda calculus as:

$$T \coloneqq o \mid i \mid T \to T$$

That is, o is a type, i is a type and given two types A and B, the expression $A \to B$ is also a type. The type o represents the type of propositions, i represents the type of individuals and $A \to B$ represents the type of functions from A to B.

Some examples:

$$o \qquad o \to i \qquad o \to i \to o \qquad (o \to i) \to o$$

We consider $\to$ to be right associative, and thus $o \to i \to o$ is the same as $o \to (i \to o)$.

We define the terms of the simply typed lambda calculus (using the Bruijn levels) as:

$$t \coloneqq n \mid t\,t \mid \lambda_T\, t \mid \alpha \mid \forall_T$$

where n is a natural number, T is a type and α is one of the following symbols:

$$\alpha \coloneqq \bot \mid \top \mid \wedge \mid \vee \mid \supset$$

Some examples:

$$\lambda_i\, 0 \qquad (\lambda_{i \to o}\, 2)\, 1 \qquad \lambda_o\, 3\, \bot \qquad \lambda_{o \to o}\, \lambda_i\, 0\, \top$$

The interpretation of a term depends on a fixed number m of free variables. All the levels which are less then m represent free variables, while the others represent bound variables. For example, if the free variables are 2 (that is, the levels 0 and 1), the terms $\lambda_i\, 0$ and $(\lambda_{i \to o}\, 2)\, 1$ can be interpreted respectively as $\lambda\, x\colon i, y$ and $(\lambda\, x\colon i \to o, x)\, y$. If the free variables are 0, then $\lambda_i\, 0$ corresponds to $\lambda\, x\colon i, x$, while the term $(\lambda_{i \to o}\, 2)\, 1$ makes no sense, because 1 and 2 should be bound variables.

1.2 Typing

Given a list of types Γ, a natural number n and a type T, we denote the n-th element of Γ by Γ_n and the list constructed by appending the element T to the end of Γ as Γ, T.

Definition 1. Given a list of types Γ, a term t and a type T, we define the

proposition $\Gamma \vdash t : T$ by the following inference rules:

$$\frac{}{\Gamma \vdash n : T} \quad \text{if } \Gamma_n = T$$

$$\frac{\Gamma \vdash s : A \to B \qquad \Gamma \vdash t : A}{\Gamma \vdash s\,t : B}$$

$$\frac{\Gamma, A \vdash t : B}{\Gamma \vdash \lambda_A\, t : A \to B}$$

$$\frac{}{\Gamma \vdash \bot : o} \qquad \frac{}{\Gamma \vdash \top : o}$$

$$\frac{}{\Gamma \vdash \wedge : o \to o \to o} \qquad \frac{}{\Gamma \vdash \vee : o \to o \to o}$$

$$\frac{}{\Gamma \vdash \supset : o \to o \to o}$$

$$\frac{}{\Gamma \vdash \forall_T : (T \to o) \to o}$$

If the proposition $\Gamma \vdash t : T$ holds, we say that t *has type* T in the *context* Γ.

Some examples:

$$\frac{}{[i;\, o] \vdash 0 : i}$$

$$\cfrac{\cfrac{\cfrac{\cfrac{}{[i \to o;\, i] \vdash 0 : i \to o} \qquad \cfrac{}{[i \to o;\, i] \vdash 1 : i}}{[i \to o;\, i] \vdash 0\,1 : o}}{i \to o \vdash \lambda_i\, 0\,1 : i \to o}}{\vdash \lambda_{i \to o}\, \lambda_i\, 0\,1 : (i \to o) \to i \to o}$$

1.3 Substitution

The substitution of the list of term s with context m into the term t, indicated as $t\,[s]_m$, is defined recursively as follows:

$$\begin{aligned}
n\,[s]_m &:= s_n \quad \text{if } n \in \mathbb{N} \text{ and } n < |s| \\
n\,[s]_m &:= n \quad \text{if } n \in \mathbb{N} \text{ and } n \geq |s| \\
(t_1\, t_2)\,[s]_m &:= (t_1\,[s]_m)\,(t_2\,[s]_m) \\
(\lambda_T\, t)\,[s]_m &:= \lambda_T\,(t\,[s_+]_m)
\end{aligned}$$

Some examples:

$$\begin{aligned}
(\lambda_o\, 1\, 0)\,[\lambda_i\, 1;\, 0]_1 &= \lambda_o\, 0\,(\lambda_i\, 2) \\
(\lambda_o\, 1\, 0)\,[\lambda_i\, 1;\, 0]_2 &= \lambda_o\, 0\,(\lambda_i\, 1) \\
(\lambda_o\, 0)\,[\lambda_i\, 0\, 1]_2 &= \lambda_o\, \lambda_i\, 0\, 1 \\
(\lambda_o\, 0)\,[\lambda_i\, 0\, 1]_1 &= \lambda_o\, \lambda_i\, 0\, 2 \\
(\lambda_o\, 0)\,[\lambda_i\, 0\, 0]_0 &= \lambda_o\, \lambda_i\, 1\, 1 \\
(\lambda_o\, 1\, 0)\,[\top]_0 &= \lambda_o\,((1\, 0)\,[\top]_0) = \lambda_o\, 1\, \top \\
(\lambda_o\, 1\, 0)\,[\lambda_i\, 0;\, 1]_2 &= \lambda_o\,((1\, 0)\,[\lambda_i\, 0;\, 1]_2) = \lambda_o\, 1\,(\lambda_i\, 0)
\end{aligned}$$

In a substitution $t\,[s]_m$, the resulting term and all the terms in the list s are meant to be interpreted with m free variables, while the term t is meant to be interpreted with a number of free variables equal to the length of s.

2 G Logic

Given a list of types σ and a term t, the expression $\sigma \triangleright t$ is called a *judgement*. Let Σ be a list of types, Γ a list of judgements and B a judgement. The expression $\Sigma; \Gamma \longrightarrow B$ is called a *sequent*. We denote the list constructed by appending a head element B to a tail Γ as B, Γ. The concatenation of two lists Σ and σ is denoted by $\Sigma\,\sigma$.

We define a calculus with the following inference rules:

$$\frac{\Sigma\,\sigma \vdash t : o}{\Sigma; [\sigma \triangleright t] \longrightarrow \sigma \triangleright t}\ \text{init} \qquad \frac{\Sigma; \Delta \longrightarrow B \qquad \Sigma; B, \Gamma \longrightarrow C}{\Sigma; \Delta\,\Gamma \longrightarrow C}\ \text{cut}$$

$$\frac{\Sigma; B, B, \Gamma \longrightarrow C}{\Sigma; B, \Gamma \longrightarrow C}\ cL \qquad \frac{\Sigma\,\sigma \vdash t : o \qquad \Sigma; \Gamma \longrightarrow C}{\Sigma; \sigma \triangleright t, \Gamma \longrightarrow C}\ wL$$

$$\frac{\Sigma\,\sigma \vdash t : o}{\Sigma; [\rho \triangleright \bot] \longrightarrow \sigma \triangleright t}\ \bot L \qquad \frac{}{\Sigma; [\,] \longrightarrow \sigma \triangleright \top}\ \top R$$

$$\frac{\Sigma\,\sigma \vdash s : o \qquad \Sigma; \sigma \triangleright t, \Gamma \longrightarrow B}{\Sigma; \sigma \triangleright t \wedge s, \Gamma \longrightarrow B}\ \wedge L \qquad \frac{\Sigma\,\sigma \vdash t : o \qquad \Sigma; \sigma \triangleright s, \Gamma \longrightarrow B}{\Sigma; \sigma \triangleright t \wedge s, \Gamma \longrightarrow B}\ \wedge L$$

$$\frac{\Sigma; \Gamma \longrightarrow \sigma \triangleright s \qquad \Sigma; \Gamma \longrightarrow \sigma \triangleright t}{\Sigma; \Gamma \longrightarrow \sigma \triangleright s \wedge t}\ \wedge R$$

$$\frac{\Sigma; \sigma \triangleright s, \Gamma \longrightarrow B \qquad \Sigma; \sigma \triangleright t, \Gamma \longrightarrow B}{\Sigma; \sigma \triangleright s \vee t, \Gamma \longrightarrow B}\ \vee L$$

$$\frac{\Sigma\,\sigma \vdash t : o \qquad \Sigma; \Gamma \longrightarrow \sigma \triangleright s}{\Sigma; \Gamma \longrightarrow \sigma \triangleright s \vee t}\ \vee R \qquad \frac{\Sigma\,\sigma \vdash s : o \qquad \Sigma; \Gamma \longrightarrow \sigma \triangleright t}{\Sigma; \Gamma \longrightarrow \sigma \triangleright s \vee t}\ \vee R$$

$$\frac{\Sigma; \Gamma \longrightarrow \sigma \triangleright s \qquad \Sigma; \sigma \triangleright t, \Gamma \longrightarrow B}{\Sigma; \sigma \triangleright s \supset t, \Gamma \longrightarrow B}\ \supset L \qquad \frac{\Sigma; \sigma \triangleright s, \Gamma \longrightarrow \sigma \triangleright t}{\Sigma; \Gamma \longrightarrow \sigma \triangleright s \supset t}\ \supset R$$

$$\frac{\Sigma\,\sigma \vdash t : T \qquad \Sigma; \sigma \triangleright s\,[0;\, 1;\, \ldots;\, |\Sigma\,\sigma|;\, t]_{|\Sigma\,\sigma|}, \Gamma \longrightarrow B}{\Sigma; (\sigma \triangleright \forall_T\, s), \Gamma \longrightarrow B}\ \forall L$$

References

[1] Alonzo Church. "A Formulation of the Simple Theory of Types". In: *The Journal of Symbolic Logic* (1940). URL: https://www.jstor.org/stable/2266170.

[2] Benjamin C. Pierce et al. *Software Foundations, Volume 1: Programming Language Foundations*. 2026. URL: https://softwarefoundations.cis.upenn.edu/plf-current.